Report on unified RL implementation with different algorithms

This report would describe the design and concepts behind the reinforcement learning (RL)-and its reward approach, which have used to dynamically adjusts transmission parameters (QoS, interval, criticality) in a hybrid CoAP-MQTT IoT architecture. The primary goal is to let the agent learn how to make decisions on its own from incentive feedback without the need for manual overriding of actions of the imposition of rigid restriction while optimizing data delivery reliability and energy efficiency.

According to thesis outline, IoT devices operate in varying network and energy conditions. Design decisions such as transmission frequency, QoS level, and whether data is critical must adapt based on:

Data Change: If sensor data changes significantly, it might be worth sending more frequently.

Battery Level (B): Low power needs energy-efficient practice.

Packet Loss (L) and Delay (D): Slow or unreliable networks require higher QoS to ensure delivery.

Incorrect design leads to excessive power consumption or missed critical data. The agent tries to learn these behaviours through well-crafted rewards.

States:

Now based on the problem domain I have selected the 'States' as follows:

Factor	Meaning
Change in Data (Δ)	How big is the change in sensor data? Small, Medium, or Large?
Battery Level (B)	Is the device running low on power? Low, Medium, or High?
Packet Loss rate (L)	Are messages getting lost on the way? Low, Medium, or High loss?
Average Delay (D)	Are messages taking too long to arrive? Low, Medium, or High delay?

Change in Data (Δ_t)- How Much is Happening? Delta (Δ_t) measures the magnitude of change in sensor data, directly addressing the periodicity concern. It helps the system determine if new data is even worth transmitting.

- Small (Δ_t=0): The change is minimal ($\Delta_t < 1$), often within the range of typical sensor noise. Immediate transmission is likely unnecessary.
- Medium (Δ_t=1): A noticeable shift has occurred ($1 \leq \Delta_t < 3$), suggesting a quicker update might be beneficial.
- Large (Δ_t=2): A significant event or substantial change has been detected ($\Delta_t \geq 3$), warranting immediate transmission. This measurement is crucial for preventing unnecessary data transmissions when nothing important has changed, thereby conserving power and bandwidth.
- $\Delta_t \in \{0=\text{Small}, 1=\text{Medium}, 2=\text{Large}\}$.

Battery Level (B_t)- How Much Power is Left? This directly indicates the device's remaining energy, a critical factor for managing the overall operational lifespan of the network. Battery Level directly reflects the energy constraints highlighted in the outline.

- Low (0): The device is in a danger zone ($B_t < 0.3$), requiring urgent power conservation measures.
- Medium (1): The battery is moderately drained ($0.3 \leq B_t < 0.7$).

- High (2): There is plenty of power remaining ($B_t \geq 0.7$). The battery level directly influences the feasibility of certain actions and the long-term viability of the device within the network.
- $B_t \in \{0=Low, 1=Medium, 2=High\}$.

Packet-Loss Rate (L_t)- How Reliable is the Connection? This state measures the fraction of data packets that fail to reach their intended destination over a given window. A high loss rate signifies a struggling or unreliable network link.

- Low Loss (0): Nearly all packets are successfully delivered ($L_t < 0.1$).
- Medium Loss (1): A noticeable number of packets are being dropped ($0.1 \leq L_t < 0.3$).
- High Loss (2): The link is highly unreliable, with a significant portion of packets lost ($L_t \geq 0.3$). The packet-loss rate directly informs decisions regarding the level of reliability guarantees (Quality of Service- QoS) needed for data transmission.
- $L_t \in \{0,1,2\}$

Average Delay (D_t)- How Fast is the Network Responding? This measures the mean round-trip time for data packets, indicating the network's responsiveness. High delay means slow communication, which can be detrimental for real-time applications.

- Low Delay (0): The network is responsive enough for real-time operations ($D_t < 100$ ms).
- Medium Delay (1): There is a noticeable lag ($100 \text{ ms} \leq D_t < 500$ ms).
- High Delay (2): The delay is too significant for many real-time control loops ($D_t \geq 500$ ms).
- $D_t \in \{0,1,2\}$.

Packet Loss and Average Delay clearly indicate reliability and network responsiveness, aligning with the QoS requirements. So total possible states would be 3^4=81. Each of those 81 states succinctly captures the combined condition of "how much changed," "how much power left," "how reliable the link is," and "how fast the network responds." So, the above states defined four state parameters that would reflect the concerns the goal of the thesis outline.By discretizing these four factors into meaningful categories (low, medium, high), my system accurately captures the state of the IoT network and devices.

Actions:

Each action is a tuple (QoS level, transmission interval, criticality flag):

QoS: {0, 1, 2}

Interval: {5s, 30s, 60s}

Criticality Flag: {0 = non-critical, 1 = critical}

This leads to 3 x 3 x 2 = 18 possible actions.

Parametric Reward function design strategy:

The reward strategy R (S, A) is based on Multi-Objective Reinforcement Learning (MORL), where a parametric reward function offers multi-objective optimization by providing a flexible framework for guiding the agent's learning. My reward strategy R (S, A) is computed by combining several individual rewards and penalties:

Timeliness Reward R_{rel}: At each step t, Based on packet status:

$$R_{rel} = \begin{cases} +1.0, packet\ on\ time \\ +0.5, packet\ late \\ -1.0, packet\ dropped \end{cases}$$

Where,

- On-time: R_{rel}=1.0
- Late: R_{rel}=0.5
- Dropped: R_{rel}=−1.0

Criticality Reward R_{crit}: Check if I have chosen critical flag C matches true criticality ($true_{crit}$):

- Match: R_{crit}=1.0
- Mismatch: Rcrit=−1.0

True criticality is calculated as:

$$R_{crit} = \begin{cases} +1.0, & c_t = trueCrit \\ -1.0, & c_t \neq trueCrit \end{cases}$$

Normalized risk = (Δ risk + Battery risk + Loss risk + Delay risk + QoS risk)/5

(risks computed by parameter severity: low severity = low risk, high severity = high risk).

QoS-Match Reward R_{qos} : Match chosen QoS with ideal QoS:

Ideal QoS calculated by:

- If loss/delay high → QoS2
- If loss/delay medium → QoS1
- Otherwise → QoS0

Then:

- Match: R_{qos}=1.0
- Mismatch: R_{qos}=−1.0

It is calculated as follows:

$$R_{qos} = \begin{cases} +1.0, & q_t = "ideal"\ QoS\ for\ (loss, delay) \\ -1.0, & otherwise \end{cases}$$

Interval-Match Reward R_{int}: Interval match based on Δ (data change):

- Δ large: 5s
- Δ medium: 30s
- Δ small: 60s

Then:

- Match: R_{int}=1.0
- Mismatch: R_{int}=−1.0

It is calculated as follows:

$$R_{int} = \begin{cases} +1.0, & \tau_t = \text{"}ideal\text{"}\ interval\ for\ \Delta \\ -1.0, & otherwise \end{cases}$$

Energy Cost Penalty: I applied an energy penalty dependent on battery level B, QoS, and transmission interval:

- QoS cost (C_{qos}): normalized from 0 (QoS0) to 1 (QoS2):

$$C_{qos} = \frac{Q}{2}$$

- Interval cost (C_{int}): normalized from 0 (60s) to 1 (5s):

$$C_{int} = \frac{60-\tau}{55}$$

Where,

since according to normalize formula, $\boldsymbol{x_{norm}} = \frac{x-x_{min}}{x_{max}-x_{min}}$, where anu quality x that lives in a range [x_{min}, x_{max}] and τ as interval

- Battery factor, f_{Bt} penalizes lower battery levels more heavily:

$$f_{Bt} = 1 + \kappa(1 - B_{lvl})$$

Where

κ (kappa): A constant parameter that determines the sensitivity or weight of the battery level's impact on f_B.

B_{lvl}: Battery level

Battery continuous value ranges from 0.3 (low) to 1.1 (high), approximated from discretized battery level.

Total energy penalty:

$$Energy_{Penalty} = f_{Bt} \times (w_{\tau_{cost}}.C_{int} + w_{qos_{cost}}.C_{qos})$$

($w_{\tau_{cost}}$, $w_{qos_{cost}}$ are weights for interval and QoS cost, respectively.)

Latency Penalty: To penalize high latencies independently:

$$Latency_{Penalty} = w_{latency}.min(1, \frac{latency}{800})$$

($w_{latency}$ is the latency penalty weight.)

Final Reward Equation: Combining all terms, the complete parametric reward function is:

$$R(S,A) = w_{rel}R_{rel} + w_{crit}R_{crit} + w_{qos_match}R_{qos} + w_{int_match}R_{int} - Energy_{Penalty} - Latency_{Penalty}$$

Where weights (w) are set as hyperparameters in my code crossing over algorithms I have used, giving flexibility to how much each criterion influences the agent's decision-making. All rewards are summed, then clipped within the range [−5.0, 5.0]. For discretization, each of {Δ, B, loss, delay}is bucketed into Low (0), Med (1), High (2) by predefined thresholds and it converts continuous states into discrete levels.

This parametric strategy is designed to balance conflicting objectives such as data freshness versus energy saving, reliability versus resource consumption, and critical classification accuracy. The logic behind this is to encourage sending critical and timely data with appropriate QoS and frequency, to penalize mismatches in QoS/interval and criticality classification to ensure the agent learns the ideal configuration for each scenario, to discourage behaviours that drain the battery unnecessarily or result in late/dropped packets strongly.

References:

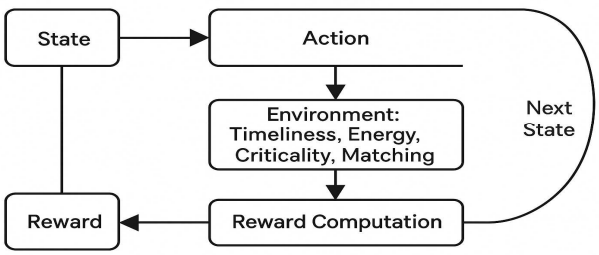
1) https://arpan-kusari.medium.com/what-happens-if-you-change-the-reward-function-after-training-in-reinforcement-learning-80bb2485e03
2) Weighted reward vector: https://www.quora.com/How-do-I-weight-reward-vector-in-reinforcement-learning
3) Composite reward: https://medium.com/@zhonghong9998/real-world-drl-5-essential-reward-functions-for-modeling-objectives-and-constraints-e742325d4747

The design of this reward function introduces a critical hyperparameter tuning challenge. The choice of weights (w_{rel}, w_{crit}, w_{qos_match}, w_{int_match}, $w_{\tau_{cost}}$, $w_{qos_{cost}}$, $w_{latency}$) for each objective is highly sensitive. In multi-objective reinforcement learning (MORL), determining these weights is proved to be a complex task, especially when objectives are inherently conflicting. Incorrectly set weights can lead to suboptimal policies that disproportionately prioritize one objective (e.g., maximizing reliability) at the expense of another (e.g., energy efficiency), potentially causing the realistic energy drain model. The battery level for agent to converge to local optima or exhibit undesirable behaviours.

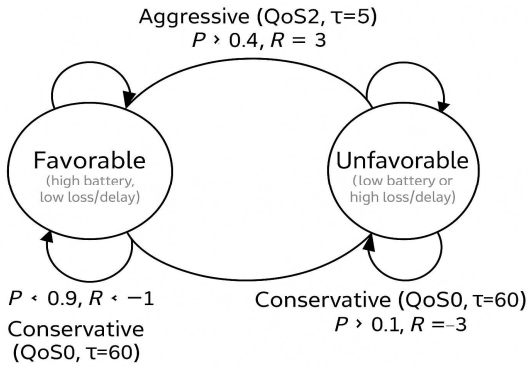
Environment simulation for testing

Since using the whole dockerized CoAP-MQTT integrated would take lot of time, so I simulate the CoAP-MQTT bridge as discrete-time by trading off QoS and transmission frequency, to classify packets as critical or non-critical based on current risk.

The agent decision block could be as follows:



Learning Feedback loop:



In this simulation, the system tracks four key state variables—sensor delta (Δ), battery level, packet loss rate, and transmission delay—each discretized into three levels (low, medium, high). The agent selects actions consisting of QoS level (0-2), transmission interval (5s, 30s, 60s), and a critical data flag (0 or 1). To efficiently evaluate communication strategies in a CoAP-MQTT hybrid IoT system, I developed a discrete-time simulation model that captures the key dynamics of sensor data transmission and network conditions. The environment maintains four primary state variables, each discretized into three levels (low, medium, high):

- Sensor Data Change (Δ): it measures the magnitude of change in sensor readings compared to a smoothed historical average using an exponential moving average (EMA). This quantifies whether new data reflects a significant event or minor fluctuation. From the raw sensor input and Δ (data change) calculation, suppose I have new sampled temperature x_t is sampled uniformly in [10, 30] ^{oC}. I compute the exponential moving average (EMA) to smooth sensor delta calculations:

  $$EMA_t = \alpha x_t + (1-\alpha)\, EMA_{t-1}, \alpha = 0.1$$

  Where $\Delta = |x_t - EMA_t|/\sigma$ with $\sigma = 1$

  α: Exponential Moving Average (EMA) smoothing factor (value: 0.1),

  EMA_t: Current Exponential Moving Average,

  EMA_{t-1}: Previous Exponential Moving Average,

  Δ: Data change,

  σ: Standard deviation for Δ calculation (value: 1),

  In simple terms, a 'moving average' is a statistical tool to determine the direction of a trend and, to do that, it aggregates a sub-set of data points usually over a specific time period and divides the total by the number of data points in the subset to yield an average. It is called a moving average because this calculation happens recursively as the data points grow with time. Exponential Moving Average is a type of Moving Average, which applies more weight to the most recent data points than those which happened in past. In other words, it is like giving more importance to the last experience or memories than to older ones, assuming those are represented by data points. Reference: https://medium.com/analytics-vidhya/understanding-exponential-moving-averages-e3f020d9d13b and https://medium.com/@prashunjaveri/sensor-data-outlier-detection-for-software-defined-vehicles-and-robots-part-1-aff7423c761e

- Battery Level: Represents the current energy reserve of the IoT device, crucial for modelling constraints on transmission frequency and power consumption.

- Packet Loss Rate: Tracks the recent ratio of dropped messages, reflecting link reliability.

- Average Transmission Delay: Measures network responsiveness by averaging recent message delivery latencies.

The learning feedback loop: The simulation implements a discrete time learning feedback loop that drives the reinforcement learning process. The sequence of operations at each time step is as follows:

- Observation: The agent observes the current state of the environment, which includes the discretized sensor delta (Δ), current battery level, packet loss rate, and average transmission delay.
- Action Selection: Based on its current reinforcement learning policy, the agent chooses an action, which is a tuple consisting of the QoS level, transmission interval (τ), and a critical data flag.
- Environment Simulation: The environment simulates the consequences of the chosen action. This involves updating the battery level based on energy consumption, simulating network delay, and determining if a packet is lost.
- True Criticality Determination: The "true criticality" of the current situation is computed based on the aggregated risk factors.
- Packet Status and Latency: The status of the packet (on-time, late, or dropped) and the actual latency are determined based on the simulated network conditions.
- Reward Calculation: A reward is calculated using the comprehensive parametric reward function, incorporating timeliness, criticality match, QoS match, interval match, energy cost penalty, and latency penalty.
- State Transition: The environment transitions to the next state based on the updated conditions.
- RL Update: The reinforcement learning algorithm (Q-learning in this case) updates its internal policy (Q-table) based on the observed (state, action, reward, next_state) tuple. This iterative process allows the agent to continuously learn and refine its communication strategy.

While this simulation environment provides a robust and efficient testbed, it is important to acknowledge the inherent "reality gap" – the discrepancy between simulated and real-world performance. The fixed parameters used in the current simulation, such as the Exponential Moving Average (EMA) smoothing factor (α=0.1) and the specific distributions for network delay and loss, might not fully capture the extensive variability and unpredictability of real IoT environments. To enhance the generalizability and robustness of the learned policies, a future direction could involve employing techniques like "domain randomization" within the simulation. This would entail varying these parameters (e.g., battery drain coefficients, network noise levels, α values) during the training phase. By exposing the agent to a wider range of environmental conditions during training, the learned policy would become more resilient and adaptable when deployed in actual IoT scenarios, effectively bridging the reality gap. At each discrete time step, the environment receives an action tuple composed of:

- QoS Level: Specifies the desired quality of service for message transmission, affecting reliability and overhead.
- Transmission Interval: Defines how frequently sensor data is sent.
- Critical Data Flag: Indicates whether the current data packet is flagged as critical.

Based on these actions, the environment simulates:

- Energy consumption, decreasing the battery according to QoS level and transmission frequency.
- Network delay, sampled from a distribution influenced by QoS and interval parameters.
- Packet loss probability, calculated as a function of QoS and network delay.

- State updates, reflecting changes in battery, delay, loss, and sensor data change.

Additionally, a true criticality flag is computed internally based on a risk model incorporating sensor change, battery, packet loss, delay, and QoS, to represent ground truth for system monitoring. This environment model provides realistic and tractable dynamics for evaluating IoT communication behaviours without hardcoding or enforcing specific action choices, allowing the agent full autonomy in selecting transmission parameters. I used this abstraction to test tabular Q learning, DQN and PPO comparing with the other machine learning models by providing structured and flexible framework for optimizing IoT communication strategies. Eventually this simulate_enviroment() function would go through Bellman equation (Q-learning), Boltzman Exploration etc. This setup ensures realism while maintain tractability for RL testing.

Tabular Q learning implementation

I have used the same states, actions and reward strategies and at first here is the pseudocode:

```
Initialize Q-table with small random values for all state-action pairs
Initialize hyperparameters: learning rate (alpha), discount factor (gamma),
                            exploration rate (epsilon), decay rates, etc.
Initialize replay buffer

For each episode:
   Reset environment to a new randomized initial state
   Discretize observations to obtain current state S
   Initialize episode metrics (reward, accuracy, latency, etc.)

   While not done and steps < max limit:
      Choose action A from state S using epsilon-Boltzmann policy:
         - With probability epsilon: sample from softmax over Q-values
         - Otherwise: choose argmax Q(S) + small noise

      Execute action A in the environment
      Observe next state S', reward R, and system feedback
      Normalize and clip reward

      Store transition (S, A, R, S') in replay buffer
      Update Q-value using Q-learning update rule:
         Q(S, A) ← Q(S, A) + α * [R + γ * max_a' Q(S', a') - Q(S, A)]

      If batch update condition met:
         Sample mini-batch from replay buffer and update Q-values

      Update current state S ← S'
      Accumulate episode metrics
      Decay alpha and epsilon based on training steps

   After episode ends:
      Repeat training on fixed test states to improve coverage

   Record episode metrics (reward, accuracy, energy, etc.)

Plot all tracked metrics to visualize training progress and convergence
```

Action Selection Strategy: Instead of purely random selection (ε-greedy), I used a more advanced method called Boltzmann exploration. This strategy uses a "softmax" function to assign probabilities to actions based on their Q-values, making the agent more likely to choose better actions while still exploring. Exploration gradually decreases over time by reducing the “temperature” and the epsilon value. This allows the agent to shift from exploration (trying different actions) to exploitation (choosing the best-known action).

The calculate_reward() function employs a composite reward structure, combining multiple objectives to guide policy optimization.

Timeline reward (R_{rel}) is implemented as +1 for on-time packet delivery, +0.5 for late delivery, and -1 for dropped packets. This directly incentivizes reliable and prompt communication.

Criticality Reward (R_{crit}) has been implemented as +1 if the agent's predicted critical flag matches the true_crit (calculated by the environment based on overall system risk), and -1 otherwise. This encourages accurate assessment of system urgency.

QoS-Match Reward (R_{qos}) is implemented as +1 if the chosen QoS matches an "ideal QoS" (dynamically determined based on loss_lvl and delay_lvl), and -1 otherwise. This aligns the agent's behaviour with system-recommended QoS for current conditions.

Interval-Match Reward (R_{int}) is implemented as +1 if the chosen interval matches an "ideal interval" (based on delta_lvl), and -1 otherwise. This promotes optimal sampling frequency.

Energy Penalty is implemented as a continuous penalty influenced by battery level (B_{lvl}), QoS cost (C_{qos}), and interval cost (C_{int}). The battery factor $f_B=1+\kappa(1-(0.3+0.4B_{lvl}))$ modulates this penalty, where κ (kappa) is a sensitivity parameter set to 2.0. Lower battery levels or higher Δ_{lvl} (through f_B) increase the penalty for high QoS or frequent intervals.

C_{qos} is normalized as qos / 2 (0 to 1.0).

C_{int} is normalized as (60 - interval) / (60 - 5) (0.0 for 60s, 1.0 for 5s).

Latency Penalty has been implemented as an additional penalty self.reward_weights['latency_penalty'] * min (1, latency/800) where latency_penalty is 0.5. This penalizes high actual latency, independent of packet status.

The hyperparameters are:

Learning Rate (α): The learning rate starts at 0.8 and decays gradually over time using exponential decay. This controls how much new experiences override previous Q-values. A high initial value encourages faster learning early on, while decay allows for fine-tuning in later stages.

Discount Factor (γ): I set to 0.95. This determines the importance of future rewards relative to immediate ones. A high value like 0.95 ensures the agent considers long-term outcomes when updating Q-values.

Exploration Rate (ϵ): The exploration rate starts at 1.0 and decays gradually to 0.05. The agent uses a softmax (Boltzmann) exploration strategy—early in training, it explores broadly by sampling from a probability distribution over actions (weighted by Q-values). As training progresses, exploration is reduced, and the agent increasingly exploits high-value actions.

Reference: https://en.wikipedia.org/wiki/Q-learning

The Q-table is initialized with small random values between -0.1 and 0.1. Crucially, specific Q-values are biased towards 1.0 for actions corresponding to critical=1 when loss_lvl >= 1 or delay_lvl >= 1. This is a form of prior knowledge injection or shaped initialization, which can significantly accelerate initial learning by guiding the agent towards potentially beneficial actions in critical scenarios, reducing

the need for extensive random exploration to discover these vital behaviours. Reference: https://www.datacamp.com/tutorial/bellman-equation-reinforcement-learning

Initially, the agent knows nothing about the environment. The Q-table is typically initialized with zeros or small random values. Let's initialize with zeros for simplicity.

State/Action	Action A0 (Low energy)	Action A1 (High energy)
S0: (Δ=0, B=0)	0.0	0.0
S1: (Δ=0, B=1)	0.0	0.0
S0: (Δ=1, B=0)	0.0	0.0
S0: (Δ=1, B=1)	0.0	0.0

The core of Q-Learning is the Bellman update equation:

$$Q(S,A) \leftarrow Q(S,A) + \alpha \times [R + \gamma \times \max_{A'} Q(S',A') - Q(S,A)]$$

Where:

- Q (S, A): Current Q-value for taking action A in state S.
- α: Learning rate starts at 0.8 and decays with steps (say early step, use α = 0.8)
- R: Immediate reward received after taking action A in state S and transitioning to S′.
- γ: Discount factor (0.95).
- $\max_{A'} Q$ (S′, A′): The maximum Q-value for any action A′ in the next state S′. This represents the optimal value of the next state.

Let's trace a few steps:

Step 1: Beginning of Episode - Action Selection

Assume the agent starts in State S2: (Δ=1, B=0) - High Change, Low Battery.

- Current Q-values for S2: Q (S2, A0) =0.0, Q (S2, A1) =0.0.
- Since all Q-values are 0, the agent might randomly choose an action due to exploration (high ϵ). Let's say it chooses Action A1 (High Energy, Fast Update).

Step 2: Environment Interaction

The agent takes Action A1 from State S2.

- Reward (R): Since Δ=1 (High Change) and it took A1 (High Energy, Fast Update), it receives a Reward = +1.0.
- Next State (S'): Let's assume the environment transitions to State S3: (Δ=1, B=1) - High Change, High Battery (e.g., due to a brief battery recharge or the action didn't drain too much relative to observation frequency).

Step 3: Q-Table Update for (S2, A1)

Now, I update Q (S2, A1). I need $\max_{A'} Q$ (S3, A′).

- Current Q-values for S3: Q (S3, A0) = 0.0, Q (S3, A1) =0.0.
- So, maxA′Q (S3, A′) = max (0.0, 0.0) =0.0.

Applying the Q-Learning formula:

$Q(S2, A1) \leftarrow Q(S2, A1) + \alpha \times [R + \gamma \times \max_{A'} Q(S3, A') - Q(S2, A1)]$

$Q(S2, A1) \leftarrow 0.0 + 0.8 \times [1.0 + 0.95 \times 0.0 - 0.0]$

$Q(S2, A1) \leftarrow 0.0 + 0.8 \times [1.0 + 0.0 - 0.0]$

$Q(S2, A1) \leftarrow 0.0 + 0.8 \times 1.0$

$Q(S2, A1) \leftarrow 0.8$

Updated Q-Table after Step 3:

State/Action	Action A0 (Low energy)	Action A1 (High energy)
S0: (Δ=0, B=0)	0.0	0.0
S1: (Δ=0, B=1)	0.0	0.0
S0: (Δ=1, B=0)	0.0	0.8
S0: (Δ=1, B=1)	0.0	0.0

The agent now "knows" that taking A1 in S2 might lead to some positive reward.

Step 4: Next Step in Episode - Action Selection

Now the agent is in State S3: (Δ=1, B=1) - High Change, High Battery.

- Current Q-values for S3: Q (S3, A0) =0.0, Q (S3, A1) = 0.0.
- Again, due to initial zeros, let's say it randomly chooses Action A0 (Low Energy, Slow Update).

Step 5: Environment Interaction

The agent takes Action A0 from State S3.

- Reward (R): Since B=1 (High Battery) and it took A0 (Low Energy, Slow Update), it receives a Reward = +0.5.
- Next State (S'): Let's assume the environment transitions to State S1: (Δ=0,B=1) - Low Change, High Battery.

Step 6: Q-Table Update for (S3, A0)

Now, I update Q (S3, A0). I need $\max_{A'} Q(S1, A')$.

- Current Q-values for S1: Q (S1, A0) =0.0, Q (S1, A1) =0.0.
- So, $\max_{A'} Q(S1, A') = \max(0.0, 0.0) = 0.0$.

Applying the Q-Learning formula:

$Q(S3, A0) \leftarrow Q(S3, A0) + \alpha \times [R + \gamma \times \max_{A'} Q(S1, A') - Q(S3, A0)]$

$Q(S3, A0) \leftarrow 0.0 + 0.8 \times [0.5 + 0.95 \times 0.0 - 0.0]$

$Q(S3, A0) \leftarrow 0.0 + 0.8 \times [0.5 + 0.0 - 0.0]$

$Q(S3, A0) \leftarrow 0.0 + 0.8 \times 0.5$

$Q(S3, A0) \leftarrow 0.4$

Updated Q-Table after Step 6:

State/Action	Action A0 (Low energy)	Action A1 (High energy)
S0: (Δ=0, B=0)	0.0	0.0
S1: (Δ=0, B=1)	0.0	0.0
S0: (Δ=1, B=0)	0.0	0.8
S0: (Δ=1, B=1)	0.4	0.0

Subplots:

I have tried several subplots through self.metrics() to provide insights into the training process and the agent's performance.

episode_rewards: A list of the total reward accumulated in each episode. This indicates the overall learning progress and should show an increasing trend as the agent learns better policies, signifying convergence.

classification_accuracy: Likely refers to the accuracy of predicting the true_crit flag. This shows how well the agent learns to assess system criticality, directly related to the R_crit reward component.

energy_consumption: Tracks the energy used per episode. This is a critical metric for evaluating the agent's efficiency and its ability to balance performance with resource conservation. A stable or decreasing trend, especially after policy optimization, would be desirable.

on_time_rate: The percentage of packets delivered on time. Directly reflects the R_rel reward and indicates network performance and responsiveness.

drop_rate: The percentage of dropped packets. The inverse of on_time_rate (combined with late_rate), indicating system reliability.

avg_latency: Average packet latency. This metric directly ties into the latency_penalty in the reward function, and its reduction is a key objective.

action_distribution: Tracks the frequency of chosen actions. This helps understand the learned policy, e.g., if the agent favours certain QoS levels or intervals under specific conditions.

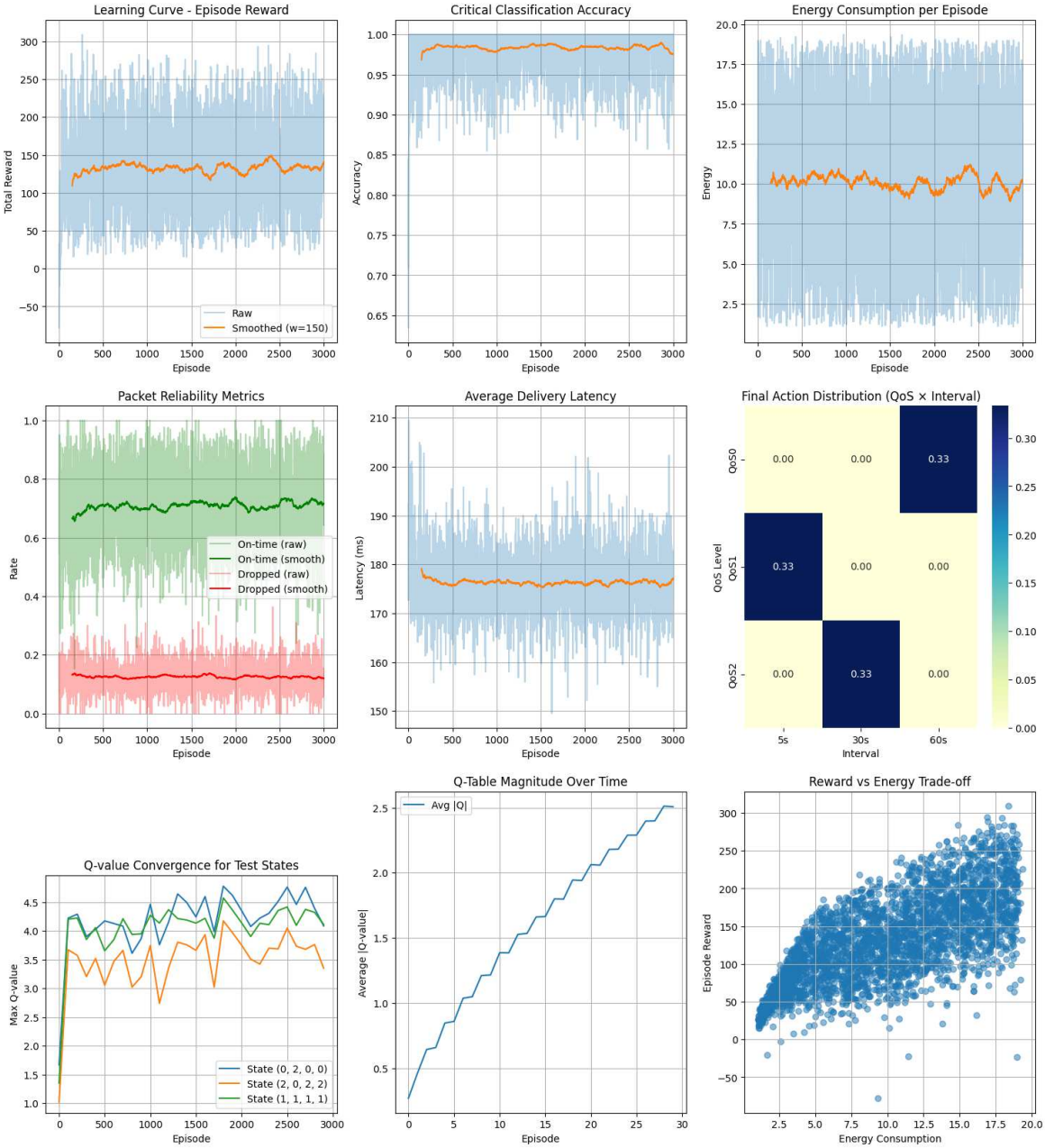
convergence_data: General data points for monitoring convergence, potentially including Q-value changes or policy stability.

q_values: Snapshots of Q-values over training. Essential for visualizing the learning process and identifying if Q-values are converging to stable estimates.

alpha_values: The learning rate (α) over episodes. Shows the decay of α, indicating the transition from aggressive updates to fine-tuning.

epsilon_values: The exploration rate (ϵ) over episodes. Shows the decay of ϵ, indicating the transition from exploration to exploitation.

Output:



Policy Analysis with Expected Ideal Actions:

State (Δ0, B=2, L=0, D=0):

Best action: QoS=0, Interval=60s, Critical=0

Q-value: 4.72

Ideal QoS: 0, Ideal Interval: 5s

Critical match: ✓

Top 1: QoS=0✓, τ=60s✗, c=0 (Q=4.72)

Top 2: QoS=1✗, τ=60s✗, c=0 (Q=2.99)

Top 3: QoS=1✗, τ=60s✗, c=1 (Q=2.96)

State (Δ2, B=0, L=2, D=2):

Best action: QoS=1, Interval=5s, Critical=1

Q-value: 4.08

Ideal QoS: 2, Ideal Interval: 5s

Critical match: ✓

Top 1: QoS=1✗, τ=5s✓, c=1 (Q=4.08)

Top 2: QoS=2✓, τ=60s✗, c=1 (Q=3.17)

Top 3: QoS=2✓, τ=30s✗, c=1 (Q=3.14)

State (Δ1, B=1, L=1, D=1):

Best action: QoS=2, Interval=30s, Critical=1

Q-value: 4.32

Ideal QoS: 1, Ideal Interval: 5s

Critical match: ✓

Top 1: QoS=2✗, τ=30s✗, c=1 (Q=4.32)

Top 2: QoS=1✓, τ=5s✓, c=1 (Q=3.61)

Top 3: QoS=0✗, τ=60s✗, c=0 (Q=3.57)

State-Action Pair Coverage Summary:

State-action pairs encountered: 1458 / 1458 (100.00%)

Well-visited pairs (≥10 visits): 1458 / 1458

Unique states visited: 81 / 81 (100.00%)

Most visited state-action pair: 50302 times

Least visited state-action pair: 14 times

Median visits per encountered pair: 14.0

Average visits per encountered pair: 160.4

5 Least-visited (state, action) pairs and counts:

State: (0, 0, 0, 0), Action: (QoS=0, Interval=5, Crit=0) --> 14 visits

State: (0, 0, 0, 0), Action: (QoS=0, Interval=5, Crit=1) --> 14 visits

State: (0, 0, 0, 0), Action: (QoS=0, Interval=30, Crit=0) --> 14 visits

State: (0, 0, 0, 0), Action: (QoS=0, Interval=30, Crit=1) --> 14 visits

State: (0, 0, 0, 0), Action: (QoS=0, Interval=60, Crit=0) --> 14 visits

Double DQN (Deep Q-Network) implementation:

I have used the same environment, states, actions, awards and episodes. I will just explain the algorithm implementation and the output.

Here is the pseudocode:

```
Initialize Policy Network (Q1) and Target Network (Q2) with same weights
Initialize Replay Buffer (D)
Initialize hyperparameters: batch_size, gamma, epsilon (start, end, decay),Exploration decay rate,
                    target_update_frequency, learning_rate, loss_function (Huberloss)
Initialize Optimizer (e.g., Adam) and learning rate scheduler

For each episode:
   Reset environment to initial state (S)
   Initialize total reward for the episode

   For each step in the episode:
      Choose action (A) from state (S) using epsilon-greedy policy on Q1:
         With probability epsilon: choose a random action
         With probability (1 - epsilon): choose action A = argmax_a Q1(S, a)

      Execute action (A) in environment:
         Observe new_state (S'), reward (R), and done signal

      Store transition (S, A, R, S', done) in replay buffer D

      S = S'
      Accumulate reward

      If replay buffer has enough samples and training:
         Sample a random batch of transitions from D
         For each transition (s, a, r, s', done) in batch:
            Calculate Target Q-value (Y_j):
               If done: Y_j = r
               Else: Y_j = r + gamma * Q2(s', argmax_a' Q1(s', a'))  (Double DQN update)

            Calculate Predicted Q-value: Q_pred = Q1(s, a)

            Calculate Loss: L = HuberLoss(Q_pred, Y_j)

            Perform backward pass and update Q1 parameters using Optimizer
```

```
    If steps_done % target_update_frequency == 0:
        Update target network parameters every few steps using a soft update: Q2 = τQ + (1- τ) Q2

    Decay epsilon

  Record episode metrics
  Update learning rate (if scheduler is used)

Plot recorded metrics
```

Double DQN Architecture uses two neural networks: policy_net (for action selection) and target_net (for Q-value evaluation). This technique addresses the overestimation bias of standard DQN, leading to more accurate Q-value estimates and improved stability. The target_net is updated periodically (target_update = 100 steps) by copying the weights from the policy_net, providing a stable target for learning.

Experience Replay Buffer uses a ReplayBuffer (with capacity=50000) stores past transitions (state, action, reward, next_state, done). During training, mini batches are sampled randomly from this buffer (batch_size = 64). This breaks temporal correlations in the data, making updates more stable and efficient, and allows for better utilization of past experiences.

Huber Loss (SmoothL1Loss) is used as nn.SmoothL1Loss() that is used as the loss function. Huber loss is less sensitive to outliers than Mean Squared Error (MSE) loss, providing a smoother gradient near the optimum and preventing large errors from dominating updates. This contributes to more stable training of the neural network.

To implement Neural Network Architecture, the DQN class defines a multi-layer perceptron (MLP) with ReLU activations.

> Dropout (0.1) layers are included after the first two ReLU activations. Dropout is a regularization technique that randomly sets a fraction of neuron activations to zero during training. This prevents overfitting by forcing the network to learn more robust features and improves generalization.

Adam Optimizer used as optim.Adam that is used for optimizing the network parameters. Adam is an adaptive learning rate optimization algorithm known for its efficiency and good performance in a wide range of deep learning tasks. The learning rate is 0.0005.

Learning Rate Scheduler is used as torch.optim.lr_scheduler.StepLR that is used to decay the learning rate. The learning rate is reduced by a gamma of 0.95 every step_size of 1000 steps. This annealing of the learning rate helps in fine-tuning the policy in later stages of training, allowing for more precise convergence.

ϵ-greedy Exploration with Exponential Decay has been implemented as the select_action method implements ϵ-greedy exploration, where ϵ decays exponentially np.exp(-1. * self.steps_done / 30000) from eps_start=0.9 to eps_end=0.05. This ensures sufficient initial exploration and then gradually reduces randomness as the agent learns, promoting exploitation of the learned policy.

State Normalization/Conversion is used as get_state_tensor that converts the discrete state tuple into a torch.FloatTensor and adds a batch dimension (unsqueeze(0)). This prepares the state for input to the neural network.

Double DQN solves this by using two separate neural networks:

> Policy Network (Q_main): This network is actively trained and is used to select the next best action (A′).
>
> Target Network (Q_target): This network is a copy of the Policy Network, but its weights are updated only periodically. It is used to evaluate the Q-value of the selected next action (Q(S′, A′)).

This decoupling ensures that the action selection (greedy choice) is not influenced by the potentially noisy Q-value estimate used for the update target, leading to more stable and accurate Q-value learning.

Let's use the same simplified environment as follows:

- States:
  - S0: (Δ=0, B=0) - Low Change, Low Battery
  - S1: (Δ=0, B=1) - Low Change, High Battery
  - S2: (Δ=1, B=0) - High Change, Low Battery
  - S3: (Δ=1, B=1) - High Change, High Battery
- Actions:
  - A0: "Low Energy, Slow Update"
  - A1: "High Energy, Fast Update"
- Hyperparameters:
  - Learning Rate (α for the optimizer): I'll conceptualize its effect on weight updates.
  - Discount Factor (γ): 0.9.
- Simplified Reward Function: (Same as previous example)
  - From (Δ, B):
    - Taking A0: +0.5 if B=1, +0.1 if B=0.
    - Taking A1: +1.0 if Δ=1, -0.5 if Δ=0.

Double DQN Training Steps:

The training process for Double DQN involves interacting with the environment, storing experiences, and then periodically sampling these experiences to update the Q_main network.

Core Update Equation for Double DQN:

The target for the network update is:

$$Yj=R+\gamma\times Q_{target}(S',\text{argmax}_{A'}Q_{main}(S',A'))$$

Where

> Y_j: The Target Q-value for the update.
>
> R: Immediate Reward.
>
> γ: Discount Factor.

Q_{target} (S′, ·): The Target Network used to evaluate the Q-value.

$\text{argmax}_{A'} Q_{main}$ (S′, A′): The action A′ that maximizes the Q-value in the next state S′, *as chosen by the Policy Network.*

Let's assume my networks currently output the following example Q-value estimates (these would change as the networks learn):

Initial Hypothetical Q-Value Estimates (Network Outputs):

State	Q_main(S, A0)	Q_main(S, A1)	Q_target(S, A0)	Q_target(S, A1)
S0 (Δ=0, B=0)	0.05	-0.02	0.05	-0.02
S1 (Δ=0, B=1)	0.01	0.01	0.10	0.01
S2 (Δ=1, B=0)	0.03	0.08	0.03	0.08
S3 (Δ=1, B=1)	0.12	0.09	0.12	0.09

(Initially, Q_main and Q_target weights are identical, hence identical outputs.)

Step 1: Agent Interaction & Experience Collection

The agent interacts with the environment using its Q_main network (e.g., ϵ-greedy policy). Let's say:

- Current State (S): S2 (Δ=1, B=0)
- Q_main(S2) outputs: [Q (S2, A0) = 0.03, Q (S2, A1) = 0.08].
- Agent chooses Action (A): A1 (since 0.08 > 0.03, assuming exploitation).
- Environment interaction: Takes A1 from S2.
    - Reward (R): +1.0 (from simplified rules: Δ=1, took A1)
    - Next State (S'): S3 (Δ=1, B=1)
    - Done: False (episode continues)

This transition (S2, A1, +1.0, S3, False) is stored in the Replay Buffer. Many such transitions are collected over time.

Step 2: Training the Policy Network (Batch Update)

After collecting enough experiences, the agent starts training by sampling mini batches from the Replay Buffer. Let's assume we sample only the transition we just collected: (S2, A1, +1.0, S3, False).

2a. Calculate the Target Q-value (Y_j):

This is the core of Double DQN. I need R + γ × Q_target(S', argmax_A' Q_main(S', A')).

- R: +1.0
- γ: 0.9
- Next State (S'): S3 (Δ=1, B=1)

Now, the Double DQN magic:

1. Use Q_main to select the best action in S':
    - Input S3 into Q_main. Let's assume Q_main(S3) still outputs [Q (S3, A0) =0.12, Q(S3, A1) = 0.09].
    - argmax_A' Q_main(S3, A') is A0 (since 0.12 is the maximum).

2. Use Q_target to evaluate the Q-value of that selected action (A0) in S':
    - Input S3 into Q_target and look up the value for A0. Let's assume Q_target(S3, A0) still outputs 0.12.

So, $\max_{A'} Q_{target}(S', A')$ is effectively Q_target(S3, A0) = 0.12.

Now, calculate Y_j:

$Y_j=1.0+0.9\times0.12$

$Y_j=1.0+0.108$

$Y_j=1.108$

Now getting the Predicted Q-value from Q_main:

- For the sampled transition (S2, A1), the predicted Q-value from Q_main is Q_main(S2, A1) = 0.08.

Calculating the Loss:

The goal is to make Q_main(S2, A1) (0.08) closer to Yj (1.108). The loss function (e.g., Huber Loss) would measure the error: Loss = Huber (1.108 - 0.08). This represents a large positive error, indicating that Q_main(S2, A1) is currently underestimating the true value.

Updating Q_main Network Weights:

The calculated loss is used to update the weights of the Q_main network through backpropagation and an optimizer (like Adam). The optimizer will adjust the weights of Q_main so that, if I were to input S2 again and ask for the Q-value of A1, it would output a value closer to 1.108.

(Conceptually, after this update, the Q_main(S2, A1) estimate might increase from 0.08 to, say, 0.15, for example).

Step 3: Periodically Update the Target Network

After a fixed number of training steps (e.g., every 100 or 1000 steps), the weights of the Q_target network are copied directly from the Q_main network.

Q_target.weights = Q_main.weights

This step is crucial for stability. It ensures that the target network remains relatively stable for a period, providing a consistent learning target for Q_main, while eventually incorporating the improvements made by Q_main.

How Q-Value Estimates "Form" Over Episodes

Random Initialization & Initial Exploration: The networks start with random weights, producing arbitrary Q-value estimates. The agent explores widely due to a high ϵ (exploration rate).

Experience Collection: As the agent explores, it collects diverse (S, A, R, S', done) transitions in the Replay Buffer. This buffer becomes a rich dataset of observed environment dynamics.

Batch Learning: Instead of learning from one experience at a time, Double DQN samples mini-batches from the buffer. This decorrelates the data, making the training of the neural networks more stable and efficient.

Iterative Refinement: With each batch update, the Q_main network's weights are adjusted to reduce the prediction error. This gradually refines its Q-value estimates. The Q_target network provides a stable, "lagging" reference for this learning.

Convergence to Optimal Policy: Over many episodes and numerous update steps:

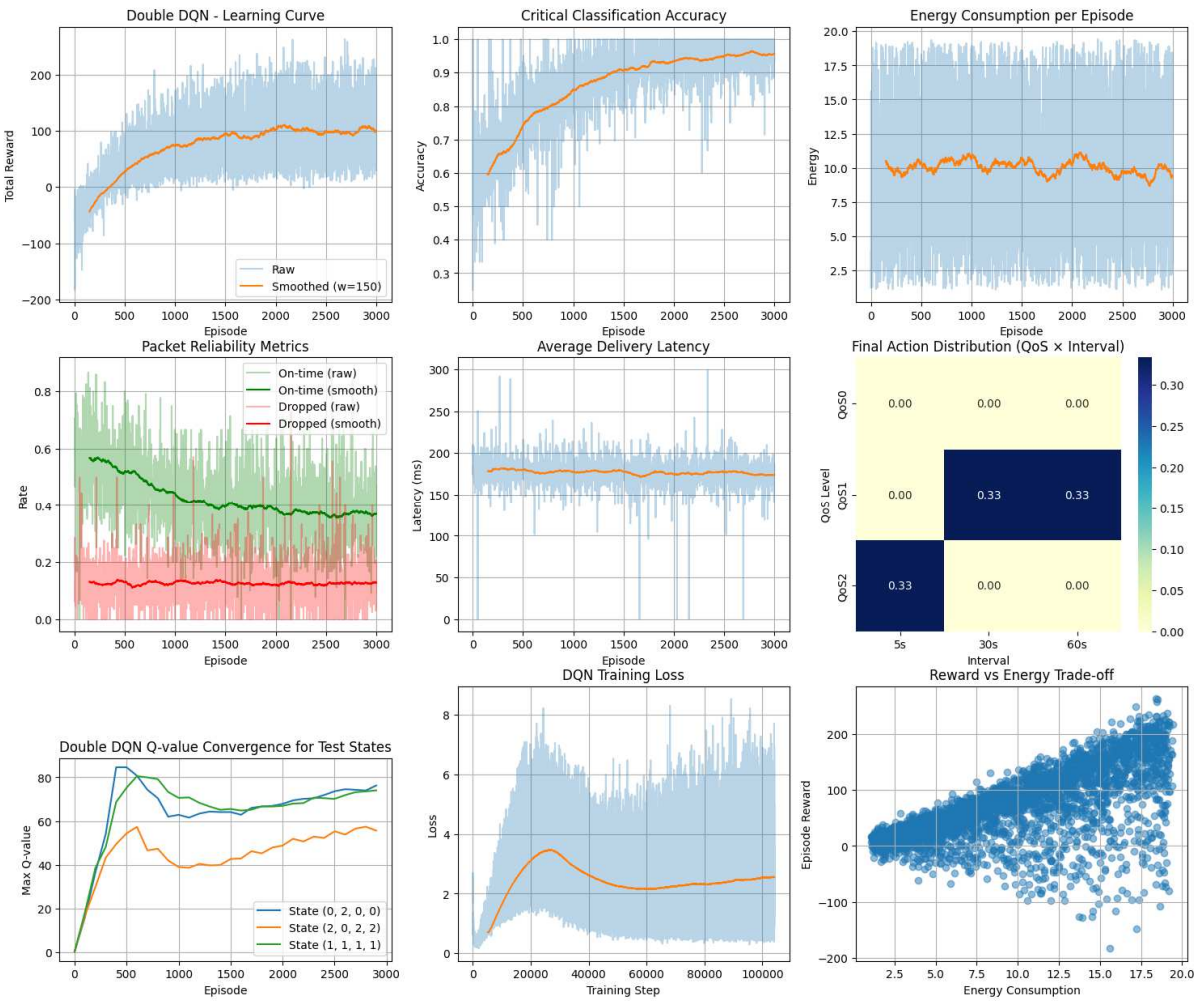
The Q_main network learns to output increasingly accurate Q-value estimates.

The overestimation bias is mitigated due to the Q_target network evaluating the actions selected by Q_main.

As Q_main's estimates become reliable, the agent's greedy policy (selecting the action with the highest Q-value output by Q_main) will converge towards the optimal policy, maximizing cumulative rewards.

The ϵ decay ensures the agent shifts from exploration to exploitation, making its behaviour more deterministic and optimal.

Output:



Double DQN Policy Analysis with Expected Ideal Actions:

State (Δ0, B=2, L=0, D=0):

Best action: QoS=1, Interval=60s, Critical=0

Ideal QoS: 0, Ideal Interval: 5s

Critical match: ✓

Top 1: QoS=1✗, τ=60s✗, c=0 (Q=76.20)

Top 2: QoS=1✗, τ=60s✗, c=1 (Q=75.09)

Top 3: QoS=2✗, τ=60s✗, c=0 (Q=74.97)

State (Δ2, B=0, L=2, D=2):

Best action: QoS=2, Interval=5s, Critical=1

Ideal QoS: 2, Ideal Interval: 5s

Critical match: ✓

Top 1: QoS=2✓, τ=5s✓, c=1 (Q=56.84)

Top 2: QoS=2✓, τ=30s✗, c=1 (Q=56.25)

Top 3: QoS=2✓, τ=60s✗, c=1 (Q=56.18)

State (Δ1, B=1, L=1, D=1):

Best action: QoS=1, Interval=30s, Critical=1

Ideal QoS: 1, Ideal Interval: 5s

Critical match: ✓

Top 1: QoS=1✓, τ=30s✗, c=1 (Q=75.73)

Top 2: QoS=1✓, τ=60s✗, c=1 (Q=72.69)

Top 3: QoS=0✗, τ=30s✗, c=0 (Q=72.40)

State-Action Pair Coverage Summary:

State-action pairs encountered: 790 / 1458 (54.18%)

Well-visited pairs (≥10 visits): 335 / 1458

Unique states visited: 81 / 81 (100.00%)

Most visited state-action pair: 16531 times

Least visited state-action pair: 1 times

Median visits per encountered pair: 5.0

Average visits per encountered pair: 131.9

5 Least-visited (state, action) pairs and counts:

State: (1, 2, 1, 2), Action: (QoS=1, Interval=60, Crit=0) --> 1 visits

State: (1, 2, 0, 2), Action: (QoS=2, Interval=60, Crit=0) --> 1 visits

State: (2, 2, 2, 1), Action: (QoS=1, Interval=60, Crit=1) --> 1 visits

State: (0, 2, 2, 2), Action: (QoS=0, Interval=60, Crit=0) --> 1 visits

State: (2, 2, 1, 0), Action: (QoS=2, Interval=5, Crit=1) --> 1 visits

Proximal Policy Optimization (PPO) Implementation:

PPO is a policy-gradient method that seeks to strike a balance between sample efficiency and ease of implementation, often outperforming DQN in continuous or high-dimensional action spaces and complex environments by directly optimizing the policy. It is renowned for its stability, achieved through a unique "clipped surrogate objective" that prevents overly aggressive policy updates, ensuring that the new policy doesn't stray too far from the old one.

Pseudocode:

```
Initialize Actor-Critic Policy Network (Policy) with random weights
Initialize PPOBuffer
Initialize hyperparameters: gamma, gae_lambda, clip_param, ppo_epochs,
                            num_minibatches, learning_rate, entropy_coef,
                            value_loss_coef, max_grad_norm
Initialize Optimizer (e.g., Adam)

For each episode or until buffer is full:
    Reset environment to initial state (S)
    Initialize total reward for the episode

    For each step in the episode (until termination or buffer capacity reached):
        Select action (A) from state (S) using Policy network (stochastic):
            Get action probabilities and state value (V) from Policy(S)
            Sample A from Categorical distribution, get log_prob(A)

        Execute action (A) in environment:
            Observe new_state (S'), reward (R), and done signal

        Store (S, A, R, S', done, log_prob(A), V) in PPOBuffer

        S = S'
        Accumulate reward

    If episode ends or buffer is full:
        Compute Advantages and Returns for collected data in PPOBuffer using GAE
        Normalize Advantages

        For ppo_epochs:
            For each minibatch sampled from PPOBuffer:
                Get states, actions, old_log_probs, returns, advantages for the batch
```

```
        Get new_action_probs, new_state_values from Policy(states)
        Compute new_log_probs from Categorical(new_action_probs)

        Calculate Ratio: ratio = exp(new_log_probs - old_log_probs)

        Calculate Surrogate Loss 1: L1 = ratio * advantages
        Calculate Surrogate Loss 2: L2 = clip(ratio, 1 - clip_param, 1 + clip_param) * advantages
        Policy Loss: L_policy = -mean(min(L1, L2))  (Clipped Surrogate Objective)

        Value Loss: L_value = MSE(new_state_values, returns)

        Entropy: H = entropy of new_action_probs
        Total Loss: L_total = L_policy + value_loss_coef * L_value - entropy_coef * H

        Perform backward pass on L_total and update Policy network parameters
        Apply gradient clipping (max_grad_norm)

    Clear PPOBuffer for next data collection phase

  Record episode metrics

Plot recorded metrics
```

Technique used for implementation:

The *ActorCritic* network architecture consists of shared layers for feature extraction followed by two separate heads: actor (outputs action probabilities) and critic (outputs state value). This allows both the policy and value function to be learned simultaneously, enabling more stable and efficient learning by providing a strong baseline for the policy gradient.

Clipped Surrogate Objective is the PPO's defining characteristic. During optimization, the policy loss function includes a clipping mechanism (clip_param = 0.10). This ensures that the ratio between the new policy's probability and the old policy's probability (ratio = exp(new_log_probs - old_log_probs)) does not deviate too far from 1. If the ratio goes beyond 1 + clip_param or below 1 - clip_param, it is clipped. This prevents destructive large policy updates, making PPO significantly more stable than other policy gradient methods.

Generalized Advantage Estimation (GAE) has been implemented as the PPOBuffer.compute_advantages() method calculates advantages using GAE. GAE is a technique to estimate the advantage function (how much better an action is than the average for a state) with reduced variance. It uses a gamma (discount factor, 0.99) and gae_lambda (0.95) to balance between bias (from short horizons) and variance (from long horizons), leading to more accurate and stable policy updates. Advantages are then normalized (advantages - advantages.mean()) / (advantages.std() + 1e-8).

Multiple Epochs and Minibatches are used as after collecting a batch of trajectories, the PPO algorithm performs multiple ppo_epochs (6 epochs) of training on num_minibatches (2 minibatches) sampled from the collected data. This multiple-pass optimization over the same data improves sample efficiency compared to on-policy methods that use data only once.

Entropy Regularization is implemented as the total loss includes an entropy bonus (entropy_coef = 0.005). This term encourages the policy to maintain a higher entropy (i.e., be more stochastic), promoting exploration and preventing the policy from collapsing into deterministic and potentially suboptimal actions too early in training.

Value Function Loss has been implemented as a separate loss term (value_loss_coef = 0.4) is added for the Critic network's prediction of the state value. This ensures that the Critic's value estimates are accurate, which is crucial for reliable advantage estimation.

max_grad_norm = 0.8 is used to clip the gradients during backpropagation. This prevents exploding gradients, a common issue in deep learning, which can destabilize training.

Adam Optimizer and Learning Rate are implemented as optim.Adam with lr = 1e-3 is used. Adam is well-suited for PPO due to its adaptive learning rate capabilities.

Replay Buffer (*PPOBuffer*): While policy gradient methods are typically "on-policy," PPO uses a buffer to store a batch of recent experiences (capacity=50000) for multiple passes during its update phase. This is distinct from DQN's experience replay, as PPO's buffer is typically cleared after each policy update.

The "Q-table" analogy doesn't directly apply here; instead, I have evolving action probabilities and state value estimates. PPO is renowned for its stability, achieved through a unique "clipped surrogate objective" that prevents overly aggressive policy updates, ensuring that the new policy doesn't stray too far from the old one.

Example Scenario: Let's use the same simplified environment as the previous implementations for consistency. Possible States are:

S0: (Δ=0,B=0) - Low Change, Low Battery

S1: (Δ=0,B=1) - Low Change, High Battery

S2: (Δ=1,B=0) - High Change, Low Battery

S3: (Δ=1,B=1) - High Change, High Battery

Actions as follows:

A0: "Low Energy, Slow Update"

A1: "High Energy, Fast Update"

Hyperparameters for PPO used in the code:

Discount Factor (γ): 0.9

GAE Lambda (λ): 0.95 (for Advantage estimation)

Clipping Parameter (ϵ_{clip}): 0.2 (This defines the "trust region" for policy updates)

Value Function Coefficient (c_v): 0.5 (Weight for value loss)

Entropy Coefficient (c_e): 0.01 (Weight for entropy bonus, encourages exploration)

- Simplified Reward Function:

  From (Δ, B):

  Taking A0: +0.5 if B=1, +0.1 if B=0.

Taking A1: +1.0 if Δ=1, -0.5 if Δ=0.

In the Actor-Critic Architecture, PPO uses two neural networks:

Actor (Policy Network): Takes a state as input and outputs a probability distribution over actions. For example, Actor(S2) might output [P(A0|S2), P(A1|S2)]. Initially, these probabilities might be close to 0.5 for each action.

Critic (Value Network): Takes a state as input and outputs a single scalar value, estimating the expected cumulative future reward from that state, V(S). For example, Critic(S2) might output 1.5.

Training involves adjusting the weights of both these networks.

PPO Training Steps:

PPO typically collects a batch of experiences (a "trajectory" or "rollout") from the environment using the current policy, then uses this data to update the policy and value networks for multiple epochs.

Step 1: Data Collection (Interacting with Environment)

The agent uses its current (random or partially trained) Actor network to select actions. For each step, it records: (State, Action, Reward, Next_State, Done, Log_Prob_of_Action, Value_from_Critic)

Let's assume I collect a very small trajectory of 2 steps,

Step a:

- Current State (S): S2 (Δ=1,B=0)
- Actor's output for S2: Let's say P(A0|S2) = 0.4, P(A1|S2) = 0.6.
- Critic's output for S2: Let's say V(S2) = 0.2 (low initial estimate).
- Action chosen by sampling: Agent samples A1.
- Log Probability of A1 (old_log_prob): ln(0.6)≈−0.51
- Environment interaction: Takes A1 from S2.
    - Reward (R): +1.0 (from simplified rules: Δ=1, took A1)
    - Next State (S'): S3 (Δ=1,B=1)
    - Done: False

Step b:

- Current State (S): S3 (Δ=1,B=1)
- Actor's output for S3: Let's say P(A0|S3) = 0.5, P(A1|S3) = 0.5.
- Critic's output for S3: Let's say V(S3) = 0.3.
- Action chosen by sampling: Agent samples A0.
- Log Probability of A0 (old_log_prob): ln(0.5)≈−0.69
- Environment interaction: Takes A0 from S3.
    - Reward (R): +0.5 (from simplified rules: B=1, took A0)

- Next State (S'): S1 (Δ=0,B=1)
- Done: False (let's assume for this example, the episode continues)

Collected Trajectory (Raw Data):

1. (S2, A1, R=1.0, S3, False, old_log_prob=-0.51, V(S2)=0.2)
2. (S3, A0, R=0.5, S1, False, old_log_prob=-0.69, V(S3)=0.3)

Step 2: Calculate Returns and Advantages

After collecting a batch of data, I calculate the "actual" returns and advantages for each step.

- Returns (R_t): The true discounted sum of future rewards from that point.
  - I need the value of the final next_state (S1 for the second step): Let's assume Critic(S1) currently outputs 0.1.
  - Return for Step 2 (S3, A0): R_2 + γ * V(S1) = 0.5 + 0.9 * 0.1 = 0.5 + 0.09 = 0.59
  - Return for Step 1 (S2, A1): R_1 + γ * Return_for_S3 = 1.0 + 0.9 * 0.59 = 1.0 + 0.531 = 1.531
- Advantages (A_t): How much better an action was than the Critic's estimate of the state's value. PPO uses Generalized Advantage Estimation (GAE).
  - TD_error = R + γ * V(S') - V(S)
  - A_t = TD_error + γ * λ * A_{t+1} (calculated backwards)

Let's simplify GAE for this small example and just calculate one-step TD error as Advantage: At =R+γV(S′)−V(S)

  - Advantage for Step 2 (S3, A0): (R=0.5) + (γ=0.9) * V(S1=0.1) - V(S3=0.3) = 0.5 + 0.09 - 0.3 = 0.29
  - Advantage for Step 1 (S2, A1): (R=1.0) + (γ=0.9) * V(S3=0.3) - V(S2=0.2) = 1.0 + 0.27 - 0.2 = 1.07

(Note: In actual PPO, Advantages are usually normalized (mean 0, std 1) to stabilize training.)

Processed Data for PPO Update:

1. (S2, A1, Return=1.531, Advantage=1.07, old_log_prob=-0.51)
2. (S3, A0, Return=0.59, Advantage=0.29, old_log_prob=-0.69)

Step 3: Policy Optimization (Actor Network Update)

PPO performs multiple "epochs" over this collected data. For each step in the batch:

- Let's focus on Step 1: (S2, A1, Return=1.531, Advantage=1.07, old_log_prob=-0.51)

1. Get New Log Probability from current Actor:
   - Input S2 into the *updated* Actor network (after some initial training).
   - Let's say the Actor now outputs P(A0|S2)=0.3, P(A1|S2)=0.7.
   - new_log_prob for A1: ln(0.7)≈−0.36

2. Calculate the Probability Ratio ($rt(\theta)$):
    - $r_t(\theta)$ = exp (new_log_prob - old_log_prob)
    - $rt(\theta)=\exp(-0.36-(-0.51)) = \exp(0.15) \approx 1.16$
    - This means the probability of choosing A1 in S2 has increased by about 16% with the new policy compared to the old one.
3. Calculate Clipped Surrogate Objective (L^{CLIP}):
    - This is the core of PPO's stability. It uses two terms and takes the minimum.
    - Term 1: $r_t(\theta)\times A_t=1.16\times 1.07=1.2412$
    - Term 2: $\text{clip}(r_t(\theta),1-\epsilon_{clip},1+\epsilon_{clip}) \times A_t$
        - clip (1.16, 1 - 0.2, 1 + 0.2) = clip (1.16, 0.8, 1.2) = 1.16 (since 1.16 is within [0.8, 1.2])
        - Term 2 = $1.16\times 1.07=1.2412$
    - Since At is positive, I want to maximize this. PPO uses -min (Term 1, Term 2) for its loss, so I calculate -1.2412.
    - What if the ratio was too large? Suppose new_log_prob made ratio = 1.3.
        - Term 1 = $1.3\times 1.07=1.391$
        - Term 2 = clip (1.3, 0.8, 1.2) = 1.2
        - Term 2 = $1.2\times 1.07=1.284$
        - Then the minimum would be 1.284. PPO would "clip" the update, effectively only considering a maximum ratio of 1.2, preventing a drastic policy change even if the unclipped ratio was higher. This prevents instability.
4. Add Entropy Bonus:
    - The entropy of the new policy's action distribution encourages exploration.
    - For [0.3, 0.7], Entropy = $-(0.3\ln(0.3) + 0.7\ln(0.7)) \approx 0.61$.
    - Entropy term: $c_e\times\text{Entropy}=0.01\times 0.61=0.0061$.
    - This term is subtracted from the total loss (because I want to maximize entropy).
5. Compute Policy Loss: The Actor's loss is the negative of the clipped objective minus the entropy bonus. This loss is then backpropagated to update the Actor network's weights.

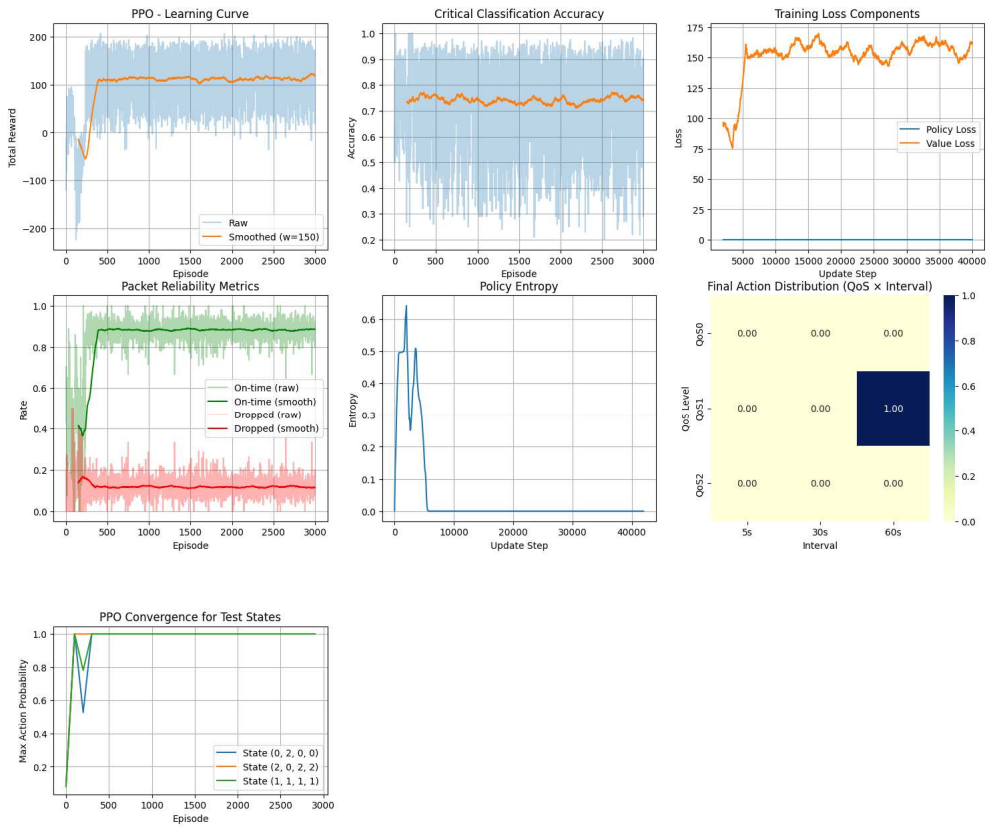
Step 4: Value Function Optimization (Critic Network Update)

Simultaneously (or sometimes in a separate pass), the Critic network is updated.

- For each step in the batch (e.g., S2, A1), I have the Return (actual target) and the Critic's predicted Value.
- Critic's output for S2: Current V(S2) = 0.2.
- Calculated Return for S2: 1.531.

- Value Loss: This is a Mean Squared Error (MSE) between the predicted value and the calculated return.
    - Loss_Value = (V(S) - Return)^2 = (0.2 - 1.531)^2 = (-1.331)^2 \approx 1.77
- This Loss_Value is scaled by cv (0.5) and then backpropagated to update the Critic network's weights. The Critic adjusts its weights to better predict the actual discounted returns.

Output:



PPO Policy Analysis with Expected Ideal Actions:

State (Δ0, B=2, L=0, D=0):

Best action: QoS=1, Interval=60s, Critical=1

Ideal QoS: 0, Ideal Interval: 5s

Critical match: ✗

Top 1: QoS=1✗, τ=60s✗, c=1 (P=1.000)

Top 2: QoS=2✗, τ=5s✓, c=1 (P=0.000)

Top 3: QoS=0✓, τ=60s✗, c=0 (P=0.000)

State (Δ2, B=0, L=2, D=2):

Best action: QoS=1, Interval=60s, Critical=1

Ideal QoS: 2, Ideal Interval: 5s

Critical match: ✓

Top 1: QoS=1✗, τ=60s✗, c=1 (P=1.000)

Top 2: QoS=2✓, τ=5s✓, c=1 (P=0.000)

Top 3: QoS=0✗, τ=60s✗, c=0 (P=0.000)

State (Δ1, B=1, L=1, D=1):

Best action: QoS=1, Interval=60s, Critical=1

Ideal QoS: 1, Ideal Interval: 5s

Critical match: ✓

Top 1: QoS=1✓, τ=60s✗, c=1 (P=1.000)

Top 2: QoS=0✗, τ=60s✗, c=0 (P=0.000)

Top 3: QoS=2✗, τ=5s✓, c=1 (P=0.000)

=== Final Metrics ===

Metric	Value
Average Reward	118.07
Accuracy	74.54%
Avg KL Divergence	0.0080
Avg Clip Fraction	1.50%
Final ε Clip	0.20
Avg Energy	7.02
Avg On-Time Rate	88.35%
Avg Drop Rate	11.65%

State-Action Pair Coverage Summary:

State-action pairs encountered: 407 / 1458 (27.91%)

Well-visited pairs (≥10 visits): 162 / 1458

Unique states visited: 81 / 81 (100.00%)

Most visited state-action pair: 51817 times

Least visited state-action pair: 1 times

Median visits per encountered pair: 5.0

Average visits per encountered pair: 581.2

5 Least-visited (state, action) pairs and counts:

State: (1, 1, 2, 0), Action: (QoS=1, Interval=60, Crit=0) --> 1 visits

State: (1, 1, 0, 1), Action: (QoS=2, Interval=60, Crit=0) --> 1 visits

State: (0, 1, 0, 1), Action: (QoS=1, Interval=30, Crit=0) --> 1 visits

State: (2, 1, 1, 1), Action: (QoS=0, Interval=30, Crit=0) --> 1 visits

State: (0, 1, 1, 1), Action: (QoS=1, Interval=60, Crit=0) --> 1 visits

The ideal sweet spot for tuning is:

Average KL Divergence: 0.005-0.008

Average Clip Fraction: 0.08-0.15

Final ε Clip: 0.07-0.10

Accuracy: >99.9%

Comparing with other machine learning models:

To implement machine learning models like logistic regression, random forest and multi-layer perceptron are used to predict optimal actions within the same *RealisticEnvSimulator* along with its states, actions and reward strategies. This predictive strategy treats action selection as a classification problem based on pre-computed "optimal" actions for given states, rather than learning through iterative interaction and maximizing long-term cumulative rewards.

The process of implementing predictive supervised learning model is quite straight forward. *update_ema(x)* and *compute_delta(x)* are calculating the Exponential Moving Average and the delta value based on a new sensor reading. *compute_loss_rate()* and *compute_avg_delay()*are used for calculating current packet loss rate and average delay based on a sliding window of historical data. *discretize_state(...)*is the crucial function that converts continuous environmental observations (delta, battery, loss, delay) into discrete levels (0, 1, 2 for Small/Low, Medium, Large/High). This results in a 4-dimensional discrete state space. *simulate_environment(state, action)* takes a current discretized state and an action (QoS, interval, critical flag) and simulates the immediate impact. It generates new raw sensor values, calculates energy cost, simulates delay and packet loss, and returns the *new_state*, *packet_status (on_time, late, dropped)*, *true_crit* (actual criticality based on a combined risk metric), energy_cost, and latency.

The *SupervisedLearningComparison* Class is the main class orchestrating the comparison.

Pseudocode:

Data Generation Process:

```
FUNCTION generate_dataset(num_episodes):
   Initialize empty lists: states_list, optimal_qos_list, optimal_interval_list, optimal_critical_list

   FOR each episode from 1 to num_episodes:
      Initialize a new RealisticEnvSimulator
      Generate random initial continuous environment values (e.g., sensor_value, battery, loss, delay)
      Discretize these values to get the current discrete state (S) = (delta_lvl, battery_lvl, loss_lvl,
delay_lvl)

      Initialize best_reward = -infinity
      Initialize best_action = None

      FOR each possible action (A_qos, A_interval, A_critical) in the Action Space (3x3x2=18
actions):
         // Simulate the environment with the current state and this specific action
         Simulate environment(S, A) to get (next_state, packet_status, true_criticality, energy_cost,
latency)
         Calculate reward (R) using calculate_reward(S, A, packet_status, true_criticality, latency)

         IF R > best_reward:
            best_reward = R
            best_action = A

      Add S to states_list
      Add best_action.qos to optimal_qos_list
      Add index of best_action.interval to optimal_interval_list (e.g., 5->0, 30->1, 60->2)
      Add best_action.critical to optimal_critical_list

   CONVERT states_list to NumPy array X
   CONVERT optimal_qos_list to NumPy array y_qos
   CONVERT optimal_interval_list to NumPy array y_interval
   CONVERT optimal_critical_list to NumPy array y_critical

   RETURN X, y_qos, y_interval, y_critical
END FUNCTION
```

A) Training Logistic Regression Model:

```
FUNCTION train_logistic_regression_models(X_train, y_qos_train, y_interval_train,
y_critical_train):

   // Model for QoS prediction

   CREATE LogisticRegression_QoS_Model

   TRAIN LogisticRegression_QoS_Model using (X_train, y_qos_train)



   // Model for Interval prediction

   CREATE LogisticRegression_Interval_Model

   TRAIN LogisticRegression_Interval_Model using (X_train, y_interval_train)
```

```
    // Model for Criticality prediction
    CREATE LogisticRegression_Critical_Model
    TRAIN LogisticRegression_Critical_Model using (X_train, y_critical_train)

    RETURN LogisticRegression_QoS_Model, LogisticRegression_Interval_Model,
LogisticRegression_Critical_Model
END FUNCTION

// In main execution:
logreg_qos_model, logreg_interval_model, logreg_critical_model =
train_logistic_regression_models(X_train, y_qos_train, y_interval_train, y_critical_train)
```

B) Training Random Forest model:

```
FUNCTION train_random_forest_models(X_train, y_qos_train, y_interval_train, y_critical_train):
    // Model for QoS prediction
    CREATE RandomForest_QoS_Model (e.g., with n_estimators=100)
    TRAIN RandomForest_QoS_Model using (X_train, y_qos_train)

    // Model for Interval prediction
    CREATE RandomForest_Interval_Model (e.g., with n_estimators=100)
    TRAIN RandomForest_Interval_Model using (X_train, y_interval_train)

    // Model for Criticality prediction
    CREATE RandomForest_Critical_Model (e.g., with n_estimators=100)
    TRAIN RandomForest_Critical_Model using (X_train, y_critical_train)

    RETURN RandomForest_QoS_Model, RandomForest_Interval_Model,
RandomForest_Critical_Model
END FUNCTION

// In main execution:
rf_qos_model, rf_interval_model, rf_critical_model = train_random_forest_models(X_train,
y_qos_train, y_interval_train, y_critical_train)
```

C) Training Multi-Layer Perception (MLP) Model (PyTorch):

```
FUNCTION train_mlp_model(X_train, y_train, output_dimension, epochs, batch_size):
    CONVERT X_train to PyTorch FloatTensor
    CONVERT y_train to PyTorch LongTensor

    CREATE PyTorch DataLoader for (X_train, y_train) with specified batch_size

    DEFINE MLP_Network:
        Input Layer (4 features -> 64 neurons)
        ReLU Activation
        Output Layer (64 neurons -> output_dimension)

    DEFINE Loss_Function = CrossEntropyLoss()
    DEFINE Optimizer = Adam(MLP_Network.parameters(), learning_rate=0.001)

    FOR epoch from 1 to epochs:
        FOR each batch (inputs, labels) from DataLoader:
            OPTIMIZER.ZERO_GRAD() // Clear gradients
            outputs = MLP_Network(inputs)
            loss = Loss_Function(outputs, labels)
            loss.BACKWARD() // Backpropagation
            OPTIMIZER.STEP() // Update network weights

    RETURN MLP_Network
END FUNCTION

// In main execution:
mlp_qos_model = train_mlp_model(X_train, y_qos_train, output_dimension=3, epochs=50,
batch_size=32)
mlp_interval_model = train_mlp_model(X_train, y_interval_train, output_dimension=3, epochs=50,
batch_size=32)
mlp_critical_model = train_mlp_model(X_train, y_critical_train, output_dimension=2, epochs=50,
batch_size=32)
```

Model Evaluation Process

```
FUNCTION evaluate_model(model_name, qos_model, interval_model, critical_model, X_test,
y_qos_test, y_interval_test, y_critical_test):
    // Make predictions
    IF model_name IS "mlp":
        CONVERT X_test to PyTorch FloatTensor
        qos_predictions = qos_model(X_test).argmax(dim=1).numpy()
        interval_predictions = interval_model(X_test).argmax(dim=1).numpy()
        critical_predictions = critical_model(X_test).argmax(dim=1).numpy()
        critical_probabilities = softmax(critical_model(X_test))[:, 1].numpy()
    ELSE (Logistic Regression or Random Forest):
        qos_predictions = qos_model.predict(X_test)
        interval_predictions = interval_model.predict(X_test)
        critical_predictions = critical_model.predict(X_test)
        critical_probabilities = critical_model.predict_proba(X_test)[:, 1]

    // Calculate classification metrics
    qos_accuracy = ACCURACY_SCORE(y_qos_test, qos_predictions)
    interval_accuracy = ACCURACY_SCORE(y_interval_test, interval_predictions)
    critical_auc = ROC_AUC_SCORE(y_critical_test, critical_probabilities)
    critical_f1 = F1_SCORE(y_critical_test, critical_predictions)

    // Calculate Average Pseudo-Reward
    Initialize pseudo_rewards_list = []
    FOR each state (S_test_i) and predicted action components (qos_pred_i, interval_pred_i,
critical_pred_i) in test set:
        Convert interval_pred_i index back to actual interval value
        Construct action_predicted = (qos_pred_i, interval_actual_i, critical_pred_i)

        // Simulate environment with the model's *predicted* action
        Simulate environment (S_test_i, action_predicted) to get (next_state, packet_status,
true_criticality, energy_cost, latency)
        Calculate reward (R) using calculate_reward(S_test_i, action_predicted, packet_status,
true_criticality, latency)
        Add R to pseudo_rewards_list
```

```
    avg_pseudo_reward = MEAN (pseudo_rewards_list)


    // Store and print all metrics
    STORE metrics in model_name's metrics dictionary
    PRINT all calculated metrics
END FUNCTION
```

In this supervised learning context, "convergence" refers to the supervised models' ability to minimize classification error, meaning they accurately predict the pre-calculated optimal actions. "Stability" implies that for similar input states, the models consistently produce similar predicted actions, leading to predictable system behaviour. "Exploration" is primarily handled by the *generate_dataset* function, which exhaustively evaluates all possible actions in a simulated environment to find the truly optimal one for each state. This provides the "ground truth" labels. The supervised models then exploit this curated dataset to learn the policy. There is no online exploration by the models themselves during their operation; they simply classify unseen states into one of the 18 pre-defined optimal actions. This approach trades the complexities of online reinforcement learning exploration for the predictability and efficiency of supervised classification.

The core of the predictive model involves generating a dataset of optimal actions and training various classifiers on this data. Dataset Generation (*generate_dataset*) creates the "expert" dataset. For *num_episodes* (e.g., 3000) randomly initialized environmental states, the system exhaustively simulates all 18 possible actions. For each state-action pair, the *simulate_environment* function provides the next state and crucial metrics (*packet_status, true_crit, latency*). The *calculate_reward* function then computes the immediate reward. The action that yields the highest reward for a given state is identified as the "optimal action" and stored along with the state as a training pair. The output is a dataset X (states), and three target vectors: *y_qos, y_interval* (indexed), and *y_critical*, enabling multi-task classification.

Training (*train_models*, *train_mlp*) generates dataset which is split into training and testing sets. Three distinct supervised learning algorithms are then trained to predict the optimal QoS, Interval, and Criticality component independently for a given state.

In Logistic Regression, model learns linear decision boundaries to classify states into the optimal QoS, Interval index, and Criticality flag. It's a fundamental linear classifier, often serving as a baseline due to its simplicity and interpretability. *max_iter*=1000 is a hyperparameter defining the maximum number of iterations for the solver to converge.

In Random Forest Classifier, ab ensemble learning method that constructs a multitude of decision trees during training and outputs the mode of the classes (for classification) of the individual trees. It's robust to overfitting, can handle non-linear relationships, and generally provides higher accuracy than single decision trees. *n_estimators*=100 is a hyperparameter indicating the number of trees in the forest.

In Multi-Layer Perceptron classifier, a fundamental neural network with one hidden layer (64 neurons) and ReLU activation, capable of learning complex, non-linear mappings between states and optimal action components. It is trained using *CrossEntropyLoss*, a standard loss function for multi-class classification, and the Adam optimizer, an adaptive learning rate optimization algorithm known for its efficiency. This model offers greater flexibility to capture intricate relationships in the data. *epochs*=50 and *batch_size*=32 are hyperparameters controlling the training iterations and batch processing.

Normalization through State Discretization: While explicit numerical normalization (e.g., min-max scaling) isn't applied to the features for the *LogisticRegression* and *RandomForestClassifier* directly, the discretization of the continuous state variables into 0, 1, 2 levels serve as an effective form of feature scaling/normalization. This ensures that all input features have a consistent range, which is beneficial for model training, especially for linear models and neural networks.

Epochs and Batch Size (MLP):

> **Epochs (50):** An epoch represents one full pass through the entire training dataset. Running for 50 epochs allows the neural network to iteratively refine its weights and biases, gradually learning the underlying patterns. A sufficient number of epochs is necessary for convergence, but too many can lead to overfitting.
>
> **Batch Size (32):** During each epoch, the training data is divided into smaller batches (here, 32 samples). The model's weights are updated after processing each batch. Using batches introduces stochasticity in the gradient descent, which can help escape local minima and often leads to faster training and better generalization than full-batch gradient descent.

Hyperparameters Definition and Role:

- *LogisticRegression(max_iter=1000)*: *max_iter* defines the maximum number of iterations for the solver. A higher value allows the model more opportunities to converge, particularly for larger or more complex datasets.
- *RandomForestClassifier(n_estimators=100): n_estimators* specifies the number of decision trees in the forest. More trees generally lead to a more robust model and better generalization, but at the cost of increased computational time.
- *MLP (nn.Linear(4, 64), lr=0.001):*

  nn.Linear(4, 64) defines the input layer (4 features, corresponding to state dimensions) and a hidden layer with 64 neurons. nn.Linear(64, output_dim) connects the hidden layer to the output layer, where output_dim is 3 for QoS/Interval and 2 for Criticality. The number of hidden layers and neurons (64 here) are hyperparameters determining the model's capacity to learn complex relationships. Too few might underfit, too many might overfit.

  lr=0.001 (learning rate for Adam optimizer) controls the step size during weight updates. An appropriate learning rate is crucial for efficient convergence without overshooting the minimum or getting stuck in suboptimal solutions. These hyperparameters are tuned to balance model complexity, training efficiency, and generalization performance.

Loss Functions:

> For the MLP models, nn.CrossEntropyLoss() is employed. This is a standard and highly effective loss function for multi-class classification problems, directly optimizing the model to produce high probabilities for the correct class (the optimal action component) and low probabilities for incorrect ones.
>
> For Logistic Regression, the training internally uses a logarithmic loss (log-loss or cross-entropy) to optimize the probability estimations.
>
> For Random Forest, the Gini impurity (or entropy) is typically used internally for splitting nodes during tree construction.

Output:

Training Logistic Regression...

LOGREG Results:

QoS Accuracy: 76.83%

Interval Accuracy: 95.17%

Critical AUC: 0.9952

Critical F1: 0.9727

Avg Pseudo-Reward: 3.93

Training Random Forest...

RANDOMFOREST Results:

QoS Accuracy: 88.33%

Interval Accuracy: 95.17%

Critical AUC: 0.9931

Critical F1: 0.9707

Avg Pseudo-Reward: 4.30

Training MLP...

MLP Results:

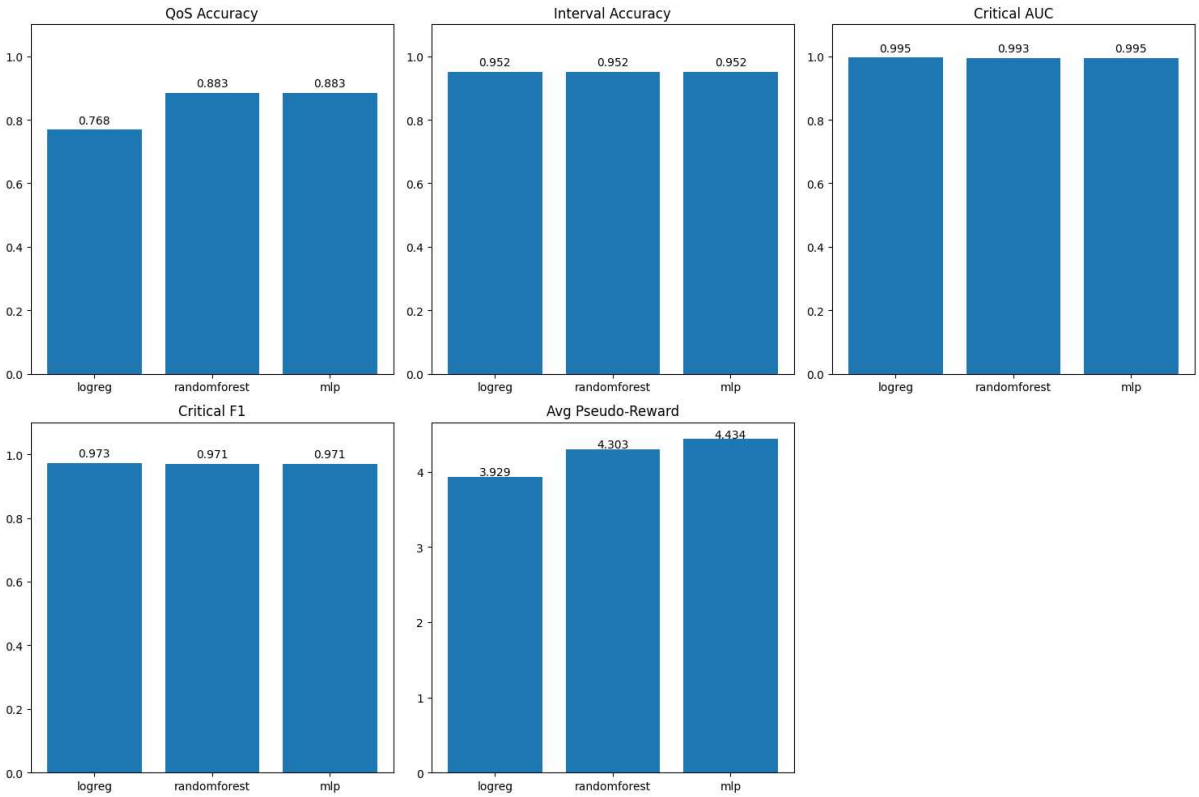
QoS Accuracy: 88.33%

Interval Accuracy: 95.17%

Critical AUC: 0.9950

Critical F1: 0.9707

Avg Pseudo-Reward: 4.43



--- Policy Analysis using model: logreg ---

State= (0, 2, 0, 0)

Model best action: QoS=0, Interval=60s, Critical=0

Ideal QoS: 0, Ideal Interval: 5s

Critical match: ✓

Top 3 model actions by probability:

1. QoS=0, Interval=60s, Critical=0 (P=0.809)
2. QoS=0, Interval=30s, Critical=0 (P=0.139)
3. QoS=1, Interval=60s, Critical=0 (P=0.044)

State= (2, 0, 2, 2)

Model best action: QoS=2, Interval=5s, Critical=1

Ideal QoS: 2, Ideal Interval: 5s

Critical match: ✓

Top 3 model actions by probability:

1. QoS=2, Interval=5s, Critical=1 (P=0.923)
2. QoS=2, Interval=30s, Critical=1 (P=0.074)
3. QoS=1, Interval=5s, Critical=1 (P=0.002)

State= (1, 1, 1, 1)

Model best action: QoS=1, Interval=30s, Critical=1

Ideal QoS: 1, Ideal Interval: 5s

Critical match: ✓

Top 3 model actions by probability:

1. QoS=1, Interval=30s, Critical=1 (P=0.391)
2. QoS=0, Interval=30s, Critical=1 (P=0.223)
3. QoS=1, Interval=30s, Critical=0 (P=0.148)

--- Policy Analysis using model: rf ---

State= (0, 2, 0, 0)

Model best action: QoS=0, Interval=30s, Critical=0

Ideal QoS: 0, Ideal Interval: 5s

Critical match: ✓

Top 3 model actions by probability:

1. QoS=0, Interval=30s, Critical=0 (P=0.605)
2. QoS=0, Interval=60s, Critical=0 (P=0.395)
3. QoS=0, Interval=5s, Critical=0 (P=0.000)

State= (2, 0, 2, 2)

Model best action: QoS=2, Interval=5s, Critical=1

Ideal QoS: 2, Ideal Interval: 5s

Critical match: ✓

Top 3 model actions by probability:

1. QoS=2, Interval=5s, Critical=1 (P=0.894)
2. QoS=2, Interval=60s, Critical=1 (P=0.106)
3. QoS=0, Interval=5s, Critical=0 (P=0.000)

State= (1, 1, 1, 1)

Model best action: QoS=1, Interval=30s, Critical=1

Ideal QoS: 1, Ideal Interval: 5s

Critical match: ✓

Top 3 model actions by probability:

1. QoS=1, Interval=30s, Critical=1 (P=0.817)

2. QoS=1, Interval=60s, Critical=1 (P=0.072)
3. QoS=0, Interval=30s, Critical=1 (P=0.050)

--- Policy Analysis using model: mlp ---

State= (0, 2, 0, 0)

Model best action: QoS=0, Interval=30s, Critical=0

Ideal QoS: 0, Ideal Interval: 5s

Critical match: ✓

Top 3 model actions by probability:

1. QoS=0, Interval=30s, Critical=0 (P=0.649)
2. QoS=0, Interval=60s, Critical=0 (P=0.350)
3. QoS=1, Interval=30s, Critical=0 (P=0.000)

State= (2, 0, 2, 2)

Model best action: QoS=2, Interval=5s, Critical=1

Ideal QoS: 2, Ideal Interval: 5s

Critical match: ✓

Top 3 model actions by probability:

1. QoS=2, Interval=5s, Critical=1 (P=0.938)
2. QoS=2, Interval=60s, Critical=1 (P=0.050)
3. QoS=2, Interval=30s, Critical=1 (P=0.012)

State= (1, 1, 1, 1)

Model best action: QoS=1, Interval=30s, Critical=1

Ideal QoS: 1, Ideal Interval: 5s

Critical match: ✓

Top 3 model actions by probability:

1. QoS=1, Interval=30s, Critical=1 (P=0.786)
2. QoS=0, Interval=30s, Critical=1 (P=0.093)
3. QoS=1, Interval=30s, Critical=0 (P=0.054)